

Course ID: SW 302

User Interface (UI) Development

Dr. Mohamed Sami Rakha

Lecture#6

Cont Tailwind , Then DOM

Lecture Outline

Dr. Mohamed Sami Rakha

- How Tailwind Generates CSS?
- How Tailwind's Layers Work?
- Styling with Utility Classes
- Plugins

Document Object Model (DOM)

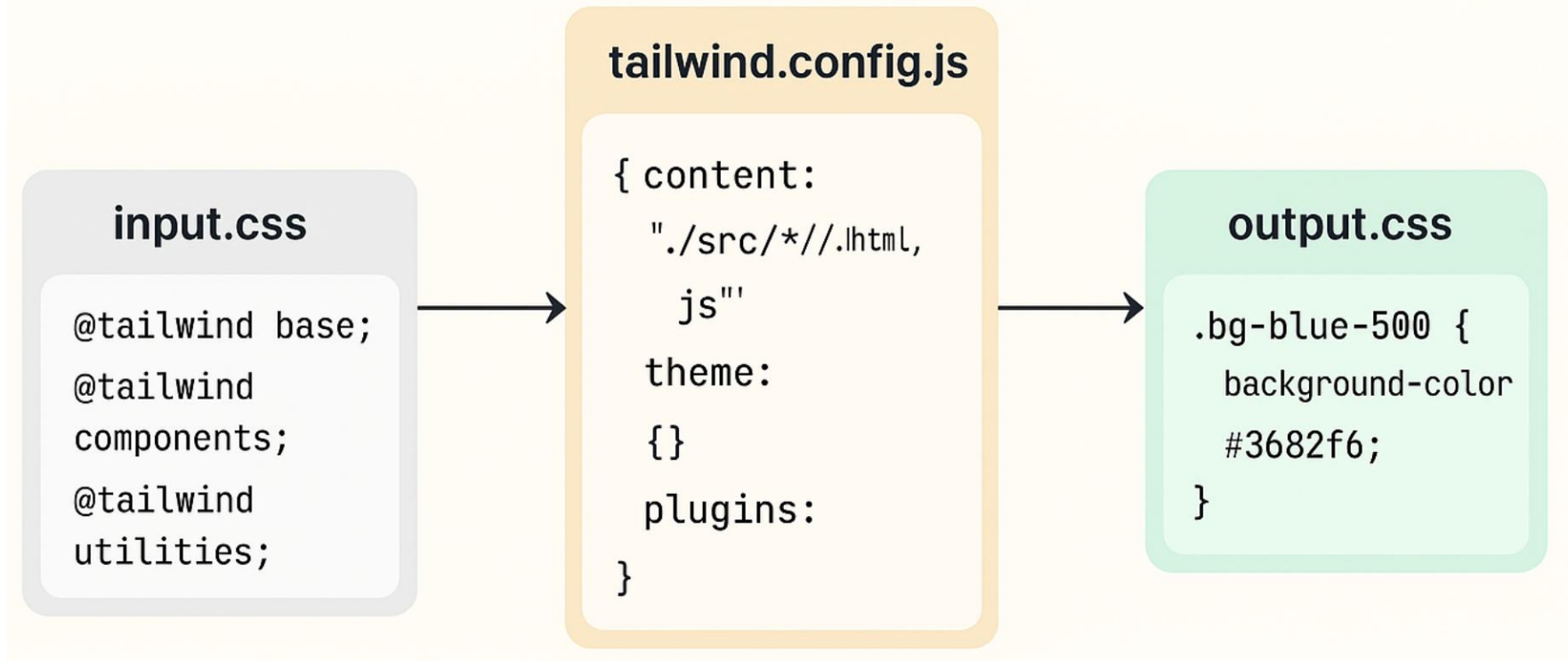
- What is DOM?
- DOM and JavaScript

Recap: How Tailwind Generates CSS?

Tailwind build process order

`npx tailwindcss -i ./src/input.css -o ./dist/output.css --watch` Dr. Mohamed Sami Rakha

The entry point for the Tailwind CLI or build tool is the **“input.css” file.**



input.css (instructions) + **tailwind.config.js** (customization) → **processed by Tailwind CLI** → **output.css** (ready-to-use file).

Tailwind build process order

This is your **source CSS file**, it tells Tailwind **what to generate**. It includes **directives** that import **Tailwind's layers**.

input.css

```
@tailwind base;  
@tailwind  
components;  
@tailwind  
utilities;
```

```
@tailwind base;  
@tailwind components;  
@tailwind utilities;
```

@tailwind base; → adds Tailwind's reset styles and base HTML elements.

@tailwind components; → includes predefined reusable classes (like buttons, forms).

@tailwind utilities; → adds all utility classes (like bg-blue-500, p-4, text-center).



You can also write your own custom CSS rules here if you want to override something.

Tailwind build process order

This file is your **Tailwind configuration**; it controls **what's included and how it looks**.

tailwind.config.js

```
{ content:
  "./src/**/*.html,
  js"
  theme:
    {}
  plugins:
}
```

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  content: ["./src/**/*.html,js"], // Files Tailwind scans to remove unused CSS
  theme: {
    extend: {
      colors: {
        brand: '#1D4ED8', // custom color
      },
    },
  },
  plugins: [],
}
```

Main parts:

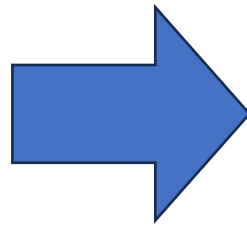
- **content:** tells Tailwind where to look for class names (HTML, JS, etc.)
- **theme:** customize default colors, fonts, spacing, breakpoints
- **extend:** safely add new styles without losing the defaults
- **plugins:** add extra functionality (like typography, forms, etc.)

Tailwind build process order

Tailwind absolutely allows you to override its default theme values, but *how* you do it depends on whether you want to **extend** or **replace** the defaults.

[1] Extend (Add to Defaults): This is the **recommended** and most common approach. It **keeps** all the built-in Tailwind styles (like **bg-blue-500**, p-4, etc.) and **adds** your custom ones **on top**.

```
module.exports = {
  theme: {
    extend: {
      colors: {
        brand: '#1D4ED8',
      },
      spacing: {
        128: '32rem',
      },
    },
  },
}
```

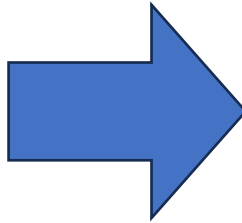


```
<div class="bg-brand p-128">Extended theme example</div>
```

All default colors, spacing, etc., still exist.

Tailwind build process order

```
// tailwind.config.js
module.exports = {
  content: ["/src/**/*.html,js"],
  theme: {
    extend: {
      colors: {
        brand: '#1E40AF', // custom blue
        accent: '#F59E0B', // warm accent color
      },
      fontFamily: {
        signature: ['Poppins', 'sans-serif'],
      },
      spacing: {
        '72': '18rem',
        '84': '21rem',
        '96': '24rem',
      },
    },
  },
  plugins: [],
}
```



```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Tailwind Theme Extension Example</title>
  <link href="output.css" rel="stylesheet">
</head>
<body class="font-signature bg-brand text-white min-h-screen flex flex-col items-center justify-center">
  <h1 class="text-4xl font-bold mb-6">Welcome to My Brand Page</h1>

  <button class="bg-accent hover:bg-yellow-500 text-white font-semibold py-3 px-6 rounded-lg shadow-md">
    Get Started
  </button>

  <div class="bg-white text-gray-700 mt-10 p-10 rounded-lg shadow-lg w-96">
    <p class="text-lg leading-relaxed">
      This box uses a <strong>custom width</strong> and <strong>brand color</strong> defined in the config
    </p>
  </div>
</body>
</html>
```

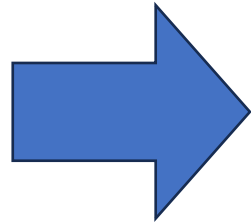
Custom Key	Utility Class Used	What It Does
brand: '#1E40AF'	bg-brand, text-brand	New background/text color
accent: '#F59E0B'	bg-accent	Accent color for buttons
fontFamily.signature	font-signature	Uses "Poppins" font
spacing['96']	w-96, h-96	Adds 24 rem spacing unit

Tailwind build process order

Tailwind absolutely allows you to override its default theme values, but *how* you do it depends on whether you want to **extend** or **replace** the defaults.

[2] Override (Replace Defaults Entirely): If you want **complete control**, you can **replace** Tailwind's defaults by **omitting** extend

```
module.exports = {  
  theme: {  
    colors: {  
      brand: '#1D4ED8',  
      white: '#FFFFFF',  
      black: '#000000',  
    },  
  },  
}
```



In this case:

Only **bg-brand**, **bg-white**, and **bg-black** will work.
All default Tailwind color classes (like **bg-blue-500**) are gone.

Tailwind build process order

This is the **final CSS file generated by Tailwind** after processing **input.css** + **tailwind.config.js**.

output.css

```
.bg-blue-500 {  
  background-color  
  #3682f6;  
}
```

```
/* Compiled Tailwind CSS */
```

```
*, ::before, ::after {  
  box-sizing: border-box;  
  border-width: 0;  
  border-style: solid;  
  border-color: #e5e7eb;  
}
```

```
/* Utilities */
```

```
.bg-blue-500 { background-color: #3b82f6; }  
.text-center { text-align: center; }  
.p-4 { padding: 1rem; }  
...
```

The `*` (asterisk) means "select everything."

Apply this rule to every element, and also to the virtual `::before` and `::after` parts of each element. The before/after control pseudo-elements like `ICONS`

How Tailwind's Layers Work?

How Tailwind's Layers Work?

Tailwind organizes all its generated CSS into **three layers**:

1. **Base**: → foundational styles
2. **Components** → reusable design patterns
3. **Utilities** → single-purpose helper classes

These layers are combined together in input.css, in this order:

```
@tailwind base;  
@tailwind components;  
@tailwind utilities;
```

Tailwind compiles layers in the following order: **base** → components → utilities

Base Layer

Provides the foundation, basic styles, and resets:

- It includes a modern CSS reset (normalize styles across browsers).
- Adds **sensible defaults for HTML** elements like headings, paragraphs, and lists.
- Often referred to as “**Preflight**” in Tailwind.
- They apply globally to elements (like setting `body { margin: 0; }`).

Where to add your own code:

If you want to **override basic HTML elements**, add your CSS **below** `@tailwind base;`.

```
@tailwind base;

h1 {
  @apply text-4xl font-bold text-gray-800;
}
```

Use this layer for
global element
styles.

Components Layer

For custom **reusable components** like buttons, cards, or alerts. These are *groups of utility classes* that you reuse across your project.

Where to add your own code: Add **after** `@tailwind components;`.

```
@tailwind base;
@tailwind components;

@layer components {
  .btn-primary {
    @apply bg-blue-600 text-white font-semibold py-2 px-4 rounded;
  }
}
```

Use this when you want to define reusable class-based components that combine multiple Tailwind utilities.



This creates a `.btn-primary` class that behaves like a Tailwind utility group.

Utilities Layer

Contains Tailwind's thousands of small, single-purpose classes, e.g. text-center, bg-red-500, p-4, etc. They are **used directly in your HTML**.

Where to add your own code: But you can your custom utility class. Add **after** @tailwind utilities;

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer utilities {
  .skew-10 {
    transform: skewY(-10deg);
  }
}
```

Use this when you need to extend Tailwind with your own small utilities.

This defines a **custom utility class** you can use like: `<div class="skew-10 bg-gray-300 p-4">Skewed box</div>`

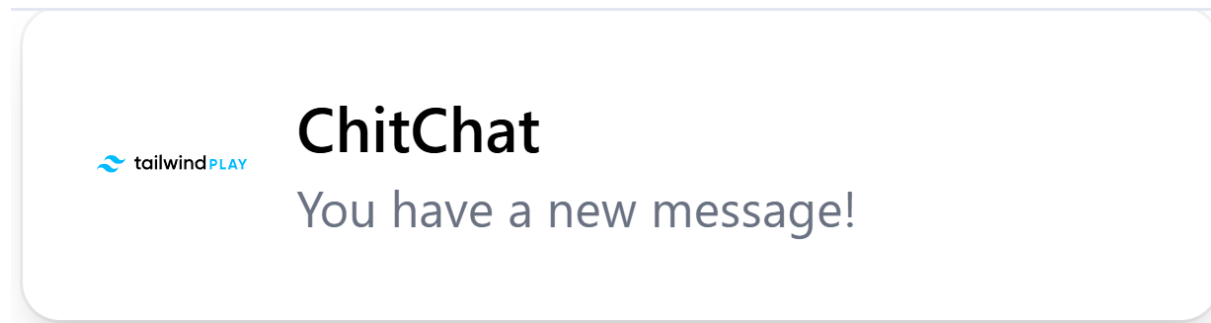


Styling with Utility Classes

How Utility Classes Help!?

You style with Tailwind is done by combining many single-purpose presentational classes (**utility classes**) directly in your markup:

```
1 <div class="mx-auto flex max-w-sm items-center gap-x-4 rounded-xl □bg-white p-6 shadow-lg  
outline □outline-black/5 ■dark:bg-slate-800 dark:shadow-none dark:-outline-offset-1  
□dark:outline-white/10">  
2     
3   <div>  
4     <div class="text-xl font-medium ■text-black □dark:text-white">ChitChat</div>  
5     <p class="■text-gray-500 □dark:text-gray-400">You have a new message!</p>  
6   </div>  
7 </div>
```



Background-color, Border-radius, and Box-shadow

```
1 <div class="mx-auto flex max-w-sm items-center gap-x-4 rounded-xl □bg-white p-6 shadow-lg  
outline □outline-black/5 ■dark:bg-slate-800 dark:shadow-none dark:-outline-offset-1  
□dark:outline-white/10">  
2     
3   <div>  
4     <div class="text-xl font-medium ■text-black □dark:text-white">ChitChat</div>  
5     <p class="■text-gray-500 ■dark:text-gray-400">You have a new message!</p>  
6   </div>  
7 </div>
```

- The **background-color**, **border-radius**, and **box-shadow** utilities (bg-white, rounded-xl, and shadow-lg) to style the card's appearance

background-color

```
.bg-white {  
background-color: var(--color-white) /*  
#fff = #ffffff */;  
}
```

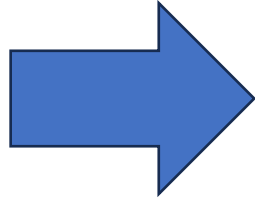
border-radius

```
.rounded-xl {  
border-radius: var(--radius-xl) /*  
0.75rem = 12px */;  
}
```

Background-color, Border-radius, and Box-shadow

CSS variables manually in @layer base

```
@layer base {  
  :root {  
    --color-white: #ffffff;  
    --color-black: #000000;  
  }  
}
```



Then in your tailwind.config.js:

```
module.exports = {  
  theme: {  
    extend: {  
      colors: {  
        white: 'var(--color-white)',  
        black: 'var(--color-black)',  
      },  
    },  
  },  
};
```

Now Tailwind will generate:

```
.bg-white {  
  background-color: var(--color-white);  
}
```

background-color

```
.bg-white {  
  background-color: var(--color-white) /*  
    #fff = #ffffff */;  
}
```

Tailwind uses `--tw-*` variables internally (like `--tw-bg-opacity`, `--tw-text-opacity`, etc.) for dynamic control.

Default Tailwind CSS output for `.bg-white`:

```
.bg-white {  
  --tw-bg-opacity: 1;  
  background-color: rgb(255 255 255 / var(--tw-bg-opacity));  
}
```

Display and Padding

```
1 <div class="mx-auto flex max-w-sm items-center gap-x-4 rounded-xl bg-white p-6 shadow-lg
  outline outline-black/5 dark:bg-slate-800 dark:shadow-none dark:-outline-offset-1
  dark:outline-white/10">
2   
3   <div>
4     <div class="text-xl font-medium text-black dark:text-white">ChitChat</div>
5     <p class="text-gray-500 dark:text-gray-400">You have a new message!</p>
6   </div>
7 </div>
```

- The **display** and **padding** utilities (flex, shrink-0, and p-6) to control the overall layout

Display

```
.flex {
  display: flex;
}

.shrink-0 {
  flex-shrink: 0;
}
```

Padding

```
.p-6 {
  padding: calc(var(--spacing) * 6) /*
    1.5rem = 24px */;
}
```

Max-width and Margin

```
1 <div class="mx-auto flex max-w-sm items-center gap-x-4 rounded-xl bg-white p-6 shadow-lg  
outline outline-black/5 dark:bg-slate-800 dark:shadow-none dark:-outline-offset-1  
dark:outline-white/10">  
2     
3   <div>  
4     <div class="text-xl font-medium text-black dark:text-white">ChitChat</div>  
5     <p class="text-gray-500 dark:text-gray-400">You have a new message!</p>  
6   </div>  
7 </div>
```

- The max-width and margin utilities (max-w-sm and mx-auto) to constrain the card width and center it horizontally

max-width

```
.max-w-sm {  
  max-width: var(--container-sm) /*  
    24rem = 384px */;  
}
```

margin

```
.mx-auto {  
  margin-inline: auto;  
}
```

Width and Height

```
1 <div class="mx-auto flex max-w-sm items-center gap-x-4 rounded-xl bg-white p-6 shadow-lg  
outline outline-black/5 dark:bg-slate-800 dark:shadow-none dark:-outline-offset-1  
dark:outline-white/10">  
2     
3   <div>  
4     <div class="text-xl font-medium text-black dark:text-white">ChitChat</div>  
5     <p class="text-gray-500 dark:text-gray-400">You have a new message!</p>  
6   </div>  
7 </div>
```

- The **width** and **height** utilities (size-12) to set the width and height of the logo image

width and height

```
.size-12 {  
  width: calc(var(--spacing) * 12) /* 3rem = 48px */;  
  height: calc(var(--spacing) * 12) /* 3rem = 48px */;  
}
```

Adding a Gap

```
1 <div class="mx-auto flex max-w-sm items-center gap-x-4 rounded-xl □bg-white p-6 shadow-lg  
outline □outline-black/5 ■dark:bg-slate-800 dark:shadow-none dark:-outline-offset-1  
□dark:outline-white/10">  
2     
3   <div>  
4     <div class="text-xl font-medium ■text-black □dark:text-white">ChitChat</div>  
5     <p class="■text-gray-500 ■dark:text-gray-400">You have a new message!</p>  
6   </div>  
7 </div>
```

- The **gap** utilities (gap-x-4) to handle the spacing between the logo and the text

Gap

```
.gap-x-4 {  
  column-gap: calc(var(--spacing) * 4) /* 1rem = 16px */;  
}
```

Font-size, Color, and Font-weight

```
1 <div class="mx-auto flex max-w-sm items-center gap-x-4 rounded-xl □bg-white p-6 shadow-lg
  outline □outline-black/5 ■dark:bg-slate-800 dark:shadow-none dark:-outline-offset-1
  □dark:outline-white/10">
2   
3   <div>
4     <div class="text-xl font-medium ■text-black □dark:text-white">ChitChat</div>
5     <p class="■text-gray-500 ■dark:text-gray-400">You have a new message!</p>
6   </div>
7 </div>
```

Rakha

- The font-size, color, and font-weight utilities (text-xl, text-black, font-medium, etc.) to style the card text

Font-size

```
.text-xl {
font-size: var(--text-xl) /* 1.25rem = 20px */;
line-height: var(--tw-leading, var(--text-xl--
line-height) /* calc(1.75 / 1.25) ≈ 1.4 */);
}
```

Color

```
.text-black {
color: var(--color-black)
/* #000 = #000000 */;
}
```

Font-Weight

```
.font-medium {
--tw-font-weight: var(--font-weight-medium) /*
500 */;
font-weight: var(--font-weight-medium) /* 500
*/;
}
@property --tw-font-weight {
syntax: "*";
inherits: false;
```

What Are Tailwind Plugins?

Where Plugins, Components, and Layers Fit

Dr. Mohamed Sami Rakha

Tailwind as having **two major** extension systems:

CSS Layers

These are written inside your CSS (@layer base, @layer components, @layer utilities).

They organize where styles live and how they cascade.

Theme/Plugins

These are defined in tailwind.config.js.

They programmatically add new utilities, components, or variants into Tailwind's engine.

What Are Tailwind Plugins?

```
// tailwind.config.js
module.exports = {
  content: ["/src/**/*.html,js"],
  theme: {
    extend: {
      colors: {
        brand: '#1E40AF', // custom blue
        accent: '#F59E0B', // warm accent color
      },
      fontFamily: {
        signature: ['Poppins', 'sans-serif'],
      },
      spacing: {
        '72': '18rem',
        '84': '21rem',
        '96': '24rem',
      },
    },
  },
  plugins: [],
}
```

Why Plugins? Three key points:

- You can create your own design utilities when Tailwind doesn't have one.
- Plugins help keep custom styles reusable and consistent.
- You can use official plugins to save time on common UI patterns.

Created also in **tailwind.config.js**

Benefits in summary:

- Add new utilities
- Register new components
- Add variants (like hover:, focus:)
- Or use official community plugins for forms, typography, etc.

What Are Tailwind Plugins?

```
const plugin = require('tailwindcss/plugin')

module.exports = {
  content: ["/src/**/*.html,js"],
  theme: {
    extend: {},
  },
  plugins: [
    plugin(function({ addUtilities }) {
      const newUtilities = {
        '.text-shadow': {
          textShadow: '2px 2px 4px rgba(0, 0, 0, 0.3)',
        },
        '.text-shadow-lg': {
          textShadow: '4px 4px 6px rgba(0, 0, 0, 0.4)',
        },
      }
      addUtilities(newUtilities, ['responsive', 'hover'])
    })
  ],
}
```

Let's say Tailwind doesn't have a **text-shadow** class. You can add it using a plugin in your `tailwind.config.js`:

You've just created your own design utility that behaves exactly like any built-in Tailwind class (including `hover:` and `md:` responsive prefixes).

Why not just use the Layer Method to add that utility class? The `@layer` method is **perfect for small, static utilities**, it's

simpler and faster to write, **Why not just use it all the time ???**

What Are Tailwind Plugins?

```
const plugin = require('tailwindcss/plugin')

module.exports = {
  content: ["/src/**/*.html,js"],
  theme: {
    extend: {},
  },
  plugins: [
    plugin(function({ addUtilities }) {
      const newUtilities = {
        '.text-shadow': {
          textShadow: '2px 2px 4px rgba(0, 0, 0, 0.3)',
        },
        '.text-shadow-lg': {
          textShadow: '4px 4px 6px rgba(0, 0, 0, 0.4)',
        },
      }
      addUtilities(newUtilities, ['responsive', 'hover'])
    })
  ],
}
```

Why not just use the Layer Method to add that utility class?

The `@layer` method is **perfect for small, static utilities**, it's simpler and faster to write. ???

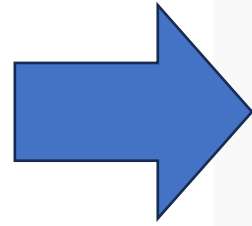
But plugins shine when you need logic or reuse.

<u>Use Case</u>	<u>Best Choice</u>
Add one or two custom classes manually	@layer utilities
Generate dozens of classes programmatically	plugins
Add responsive/hover variants automatically	plugins
Access theme values dynamically (e.g., colors, spacing)	plugins
Share your utilities as an npm package	plugins

What Are Tailwind Plugins?

Dr. Mohamed Sami Rakha

addUtilities() is a helper function provided by Tailwind's plugin API.



```
<body class="min-h-screen flex flex-col items-center justify-center bg-gray-100">
  <h1 class="text-5xl font-bold text-blue-700 text-shadow-lg">
    Tailwind Plugin Demo
  </h1>
  <p class="mt-4 text-gray-700 text-shadow">
    This text uses a custom utility defined by a plugin.
  </p>
```

Both texts have glowing shadows — created entirely by a custom plugin.

- The plugin API gives access to functions like **addUtilities**, **addComponents**, **addVariant**, etc. **Note: (JavaScript Functions).**
- Tailwind merges these automatically when building CSS.

```
const plugin = require('tailwindcss/plugin')

module.exports = {
  content: ["../src/**/*.{html,js}"],
  theme: {
    extend: {},
  },
  plugins: [
    plugin(function({ addUtilities }) {
      const newUtilities = {
        '.text-shadow': {
          textShadow: '2px 2px 4px rgba(0, 0, 0, 0.3)',
        },
        '.text-shadow-lg': {
          textShadow: '4px 4px 6px rgba(0, 0, 0, 0.4)',
        },
      }
      addUtilities(newUtilities, ['responsive', 'hover'])
    })
  ],
}
```

What Are Tailwind Plugins?

Plugins let you extend Tailwind's functionality, so you can:

```
const plugin = require('tailwindcss/plugin')

module.exports = {
  content: ["/src/**/*.html,js"],
  theme: {
    extend: {},
  },
  plugins: [
    plugin(function({ addUtilities }) {
      const newUtilities = {
        '.text-shadow': {
          textShadow: '2px 2px 4px rgba(0, 0, 0, 0.3)',
        },
        '.text-shadow-lg': {
          textShadow: '4px 4px 6px rgba(0, 0, 0, 0.4)',
        },
      }
      addUtilities(newUtilities, ['responsive', 'hover'])
    })
  ],
}
```

registers new utility classes (that you defined in newUtilities) into Tailwind's system and tells Tailwind **which variants** (like hover: or responsive breakpoints) those utilities should support.

Tailwind will generate multiple versions automatically:

```
/* Hover version */
.hover\:text-shadow:hover { ... }

/* Responsive versions */
@media (min-width: 640px) {
  .sm\:text-shadow { ... }
}
@media (min-width: 768px) {
  .md\:text-shadow { ... }
}
```

What are “variants” in Tailwind?

Variants in Tailwind determine when or how a utility applies.

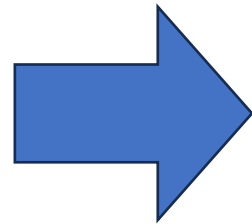
Dr. Mohamed Sami Rakha

```
const plugin = require('tailwindcss/plugin')

module.exports = {
  content: ["/src/**/*.html,js"],
  theme: {
    extend: {},
  },
  plugins: [
    plugin(function({ addUtilities }) {
      const newUtilities = {
        '.text-shadow': {
          textShadow: '2px 2px 4px rgba(0, 0, 0, 0.3)',
        },
        '.text-shadow-lg': {
          textShadow: '4px 4px 6px rgba(0, 0, 0, 0.4)',
        },
      }
      addUtilities(newUtilities, ['responsive', 'hover'])
    })
  ],
}
```

They are prefixes like:

- sm:, md:, lg: → responsive variants
- hover:, focus:, active: → state variants
- dark: → dark mode variant



```
<body class="min-h-screen flex flex-col items-center justify-center bg-gray-100">
  <h1 class="text-5xl font-bold text-blue-700 text-shadow-lg">
    Tailwind Plugin Demo
  </h1>
  <p class="mt-4 text-gray-700 text-shadow">
    This text uses a custom utility defined by a plugin.
  </p>
```

Both texts have glowing shadows — created entirely by a *custom plugin*.

Combining Everything

Plugins let you extend Tailwind's functionality, so you can:

Dr. Mohamed Sami Rakha

```
// tailwind.config.js
const plugin = require('tailwindcss/plugin')

module.exports = {
  theme: { extend: {} },
  plugins: [
    plugin(function({ addBase, addComponents, addUtilities }) {
      addBase({
        h1: { fontSize: '2.25rem', fontWeight: '700' },
      })
      addComponents({
        '.btn': { padding: '0.5rem 1rem', borderRadius: '0.5rem', fontWeight: '600' },
      })
      addUtilities({
        '.text-glow': { textShadow: '1px 1px 3px rgba(0,0,0,0.3)' },
      }, ['hover'])
    })
  ]
}
```

All in **one place**, that's the power of the plugin system.

- Tailwind's CSS layers organize your styles.
- Tailwind's plugin system *injects* new styles (base, components, utilities) into those layers, automatically and programmatically (**Javascript Functions**).

Theme Extend vs Tailwind Plugins

1. `theme.extend`: Adding or modifying design tokens

extending Tailwind's design system, *adding new values* that Tailwind will automatically generate utilities for

```
module.exports = {
  theme: {
    extend: {
      colors: {
        brand: '#1e40af', // new color
      },
      spacing: {
        '72': '18rem', // new spacing utility
      },
      fontFamily: {
        heading: ['Poppins', 'sans-serif'],
      },
    },
  },
}
```

2. `plugins`: Creating new utility types or behaviors

go **beyond what Tailwind generates automatically**.

- completely new CSS properties that Tailwind doesn't include (e.g., `text-shadow`, `clip-path`, `scroll-snap`).
- custom components (like `.btn-primary`, `.card`, `.input`).

Generate utilities programmatically (e.g., using loops, or dynamically reading from the theme).

```
plugins: [
  function({ addUtilities }) {
    const newUtilities = {
      '.text-shadow-sm': { textShadow: '1px 1px 2px rgba(0,0,0,0.25)' },
      '.text-shadow-lg': { textShadow: '3px 3px 6px rgba(0,0,0,0.4)' },
    }
    addUtilities(newUtilities, ['responsive', 'hover'])
  },
]
```

Official Tailwind Plugins

Tailwind provides some official plugins that are very useful:

➤ npm install @tailwindcss/forms

➤ Then add it to your config:

```
plugins: [  
  require('@tailwindcss/forms'),  
]
```



Now all your form elements (<input>, <select>, <textarea>) will automatically look modern and consistent.

Simple

Full name

Email address

When is your event?

What type of event is it?

Additional details

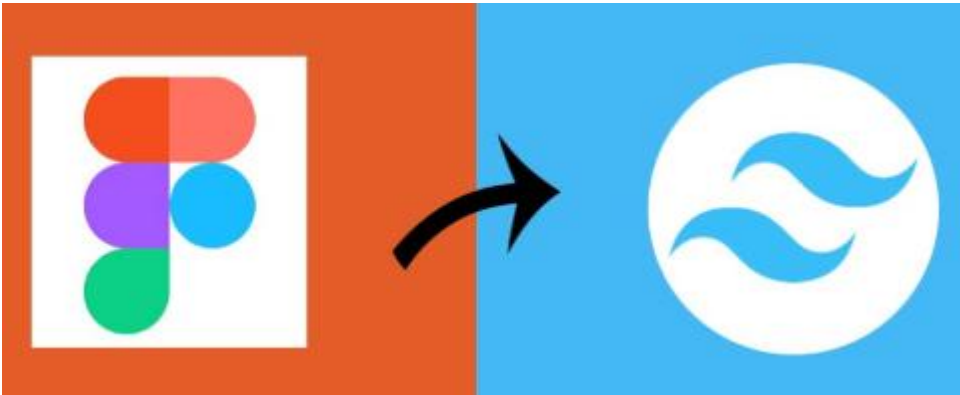
☒ Email me news and special offers

UI/UX Design to Tailwind

UI/UX Design to Tailwind



Export your design to
tailwindcss theme file



Figma Tailwindcss

	default/red/700	#c53030	Update	x
	default/red/900	#9b2c2c	Update	x
	default/red/900	#742a2a	Update	x
	default/orange/100	#ffa10	Update	x
	default/orange/200	#febcb8	Update	x

Add new color

Color name Hex code

Next

Figma Tailwindcss

Export theme

Almost there...

☒ Group colors

```
{
  "colors": {
    "shade": {
      "black": "#000000",
      "white": "#ffffff"
    },
    "gray": {
      "100": "#f7f7fc",
      "200": "#e6e6fa",
      "300": "#dcdcdc",
      "400": "#cccccc",
      "500": "#bcbcbc",
      "600": "#a9a9a9",
      "700": "#808080",
      "800": "#696969",
      "900": "#545454"
    },
    "red": {
      "100": "#ffcc99",
      "200": "#ff9966",
      "300": "#ff6633",
      "400": "#ff3300",
      "500": "#ff0000",
      "600": "#cc0000",
      "700": "#990000",
      "800": "#660000",
      "900": "#330000"
    }
  }
}
```

Figma Tailwindcss

Font sizes

Pick a base font-size

16px

12px	- 0.75rem	xs
14px	- 0.875rem	sm
16px	- 1rem	base
18px	- 1.125rem	lg
20px	- 1.25rem	xl
24px	- 1.5rem	2xl
30px	- 1.875rem	3xl
36px	- 2.25rem	4xl
48px	- 3rem	5xl
64px	- 4rem	6xl

Figma Tailwindcss

```
{
  "fontFamily": {
    "source-sans-pro": "source sans pro",
    "arial": "arial"
  }
}
```

Create file

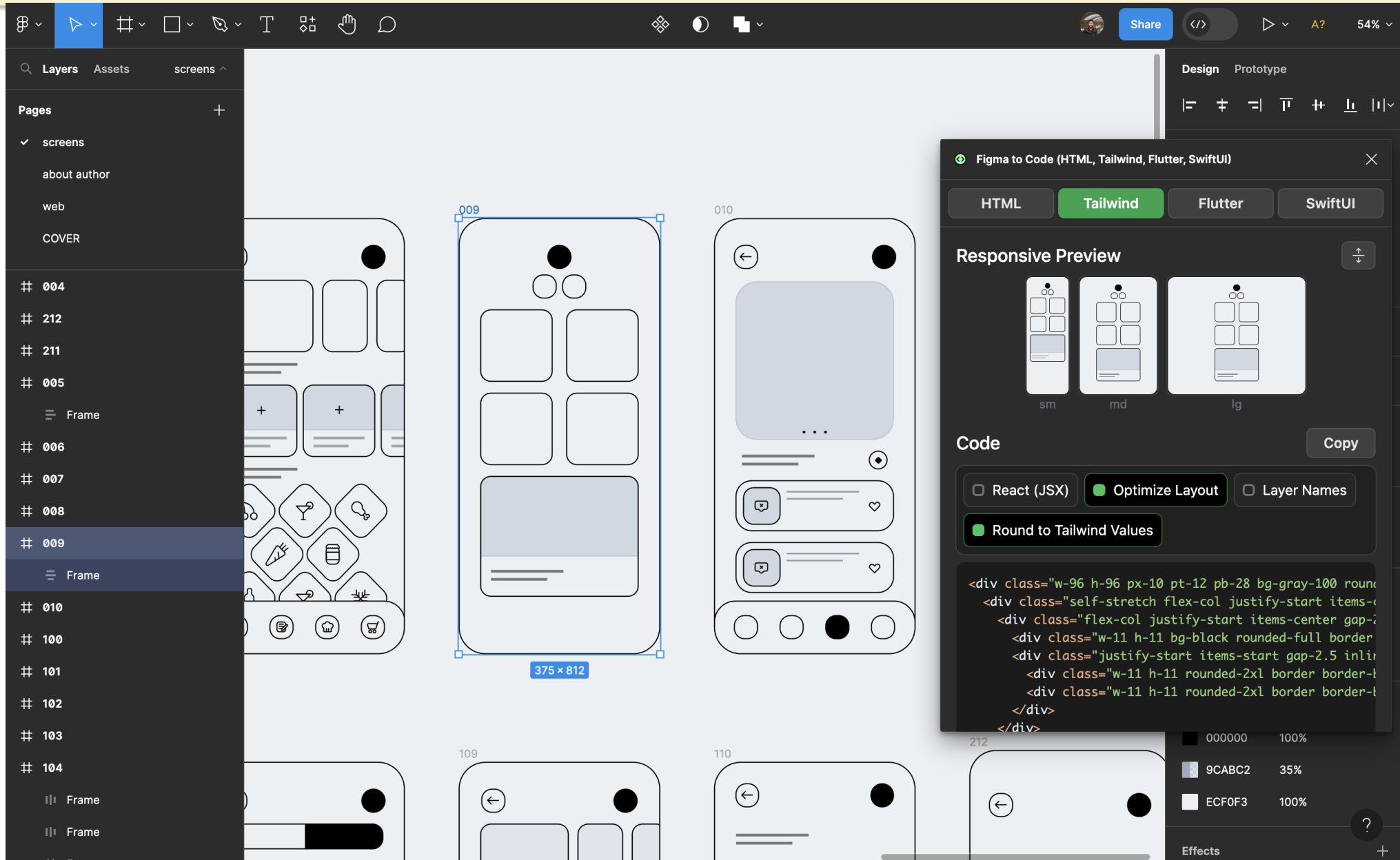
Previous

Usage

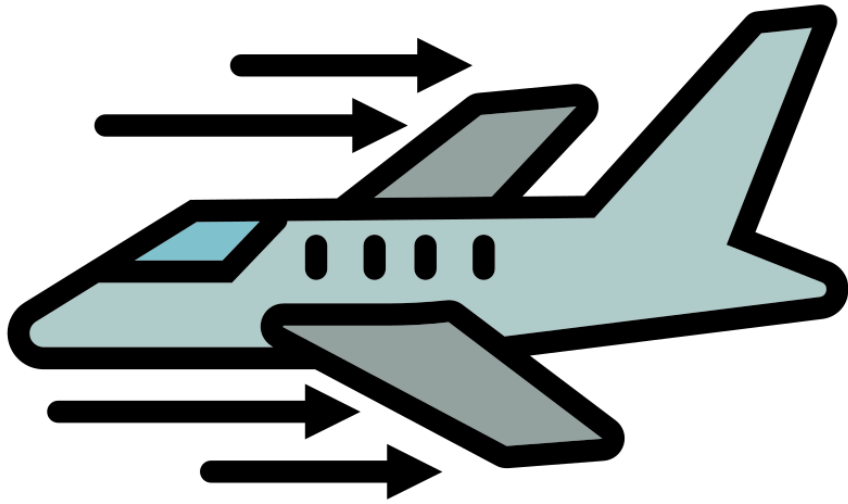
tailwindcss

GitHub

UI/UX Design to Tailwind



Dr. Mohamed Sami Rakha



Tailwind CSS

Document Object Model (DOM)

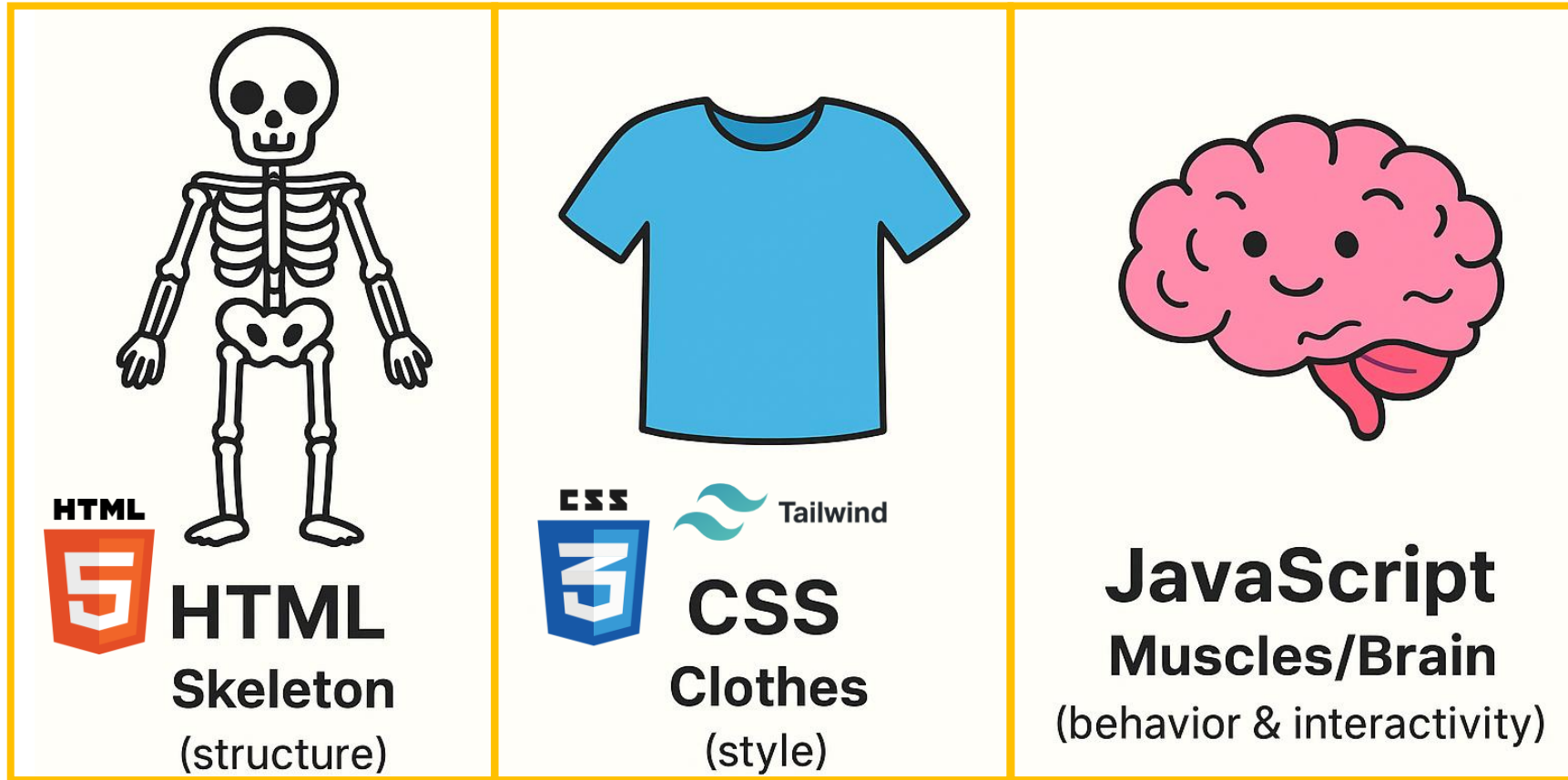
What is DOM?

The **Document Object Model (DOM)** is an API that enables programs to interact with HTML documents, where the page is represented as a tree structure, with each element, attribute, and text node forming part of the tree.

The DOM is **neither part of HTML nor part of JavaScript**; it is a separate set of rules. It is implemented by all major browser makers, and covers two primary areas:

- Making a **model** of the HTML page.
- Accessing and **changing** the HTML page.

DOM Connects all (HTML, CSS, JavaScript)



DOM

The interface that connects
HTML (structure), CSS (style),
and JavaScript (behavior)

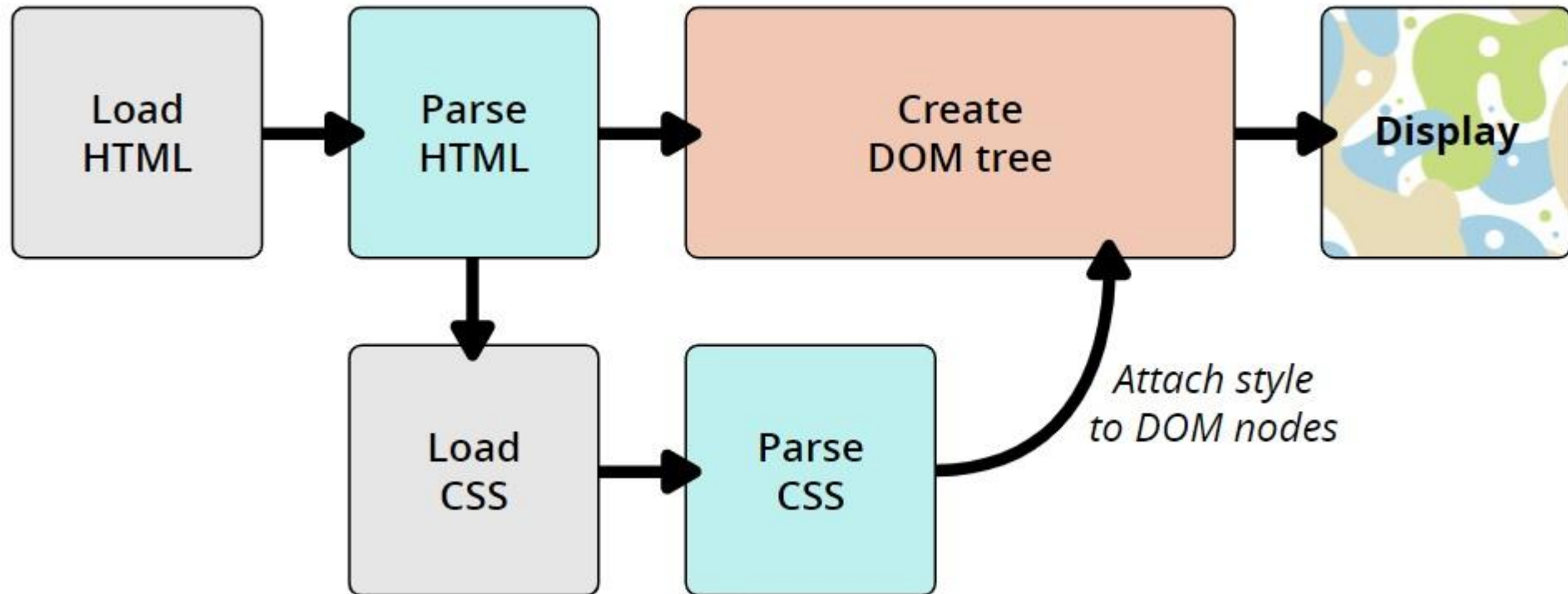
Why DOM Matters?

Dr. Mohamed Sami Rakha

- Connects **HTML** (structure), **CSS** (style), and **JS** (behavior).
- Enables dynamic web pages and interactive UIs.
- Without DOM, JS cannot “talk” to the page.

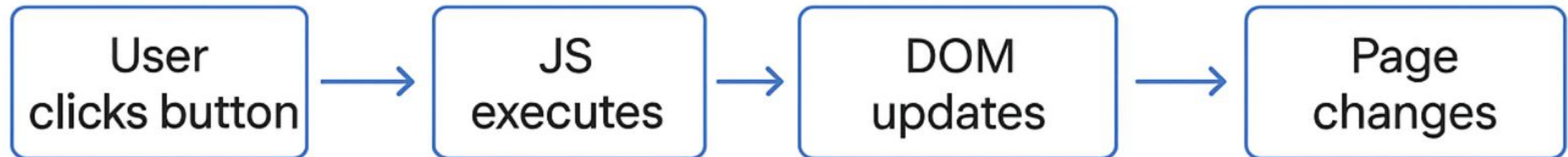
DOM Connects all

- **How does the DOM work with CSS?** CSS rules are mapped to the DOM elements. The browser combines the DOM and CSS Rules into a render tree, then paints it on the screen.



DOM Connects all

- **Connects web pages with programming languages.**
 - Lets you dynamically change the structure, style, and content.
- **Programming language neutral.**
 - Can be accessed using JavaScript, Python, Java, etc.
- **Not part of HTML or JavaScript**
- **In typical browsers, the JavaScript version of the API is provided via the **Document Host Object****



DOM Tree

Dr. Mohamed Sami Rakha

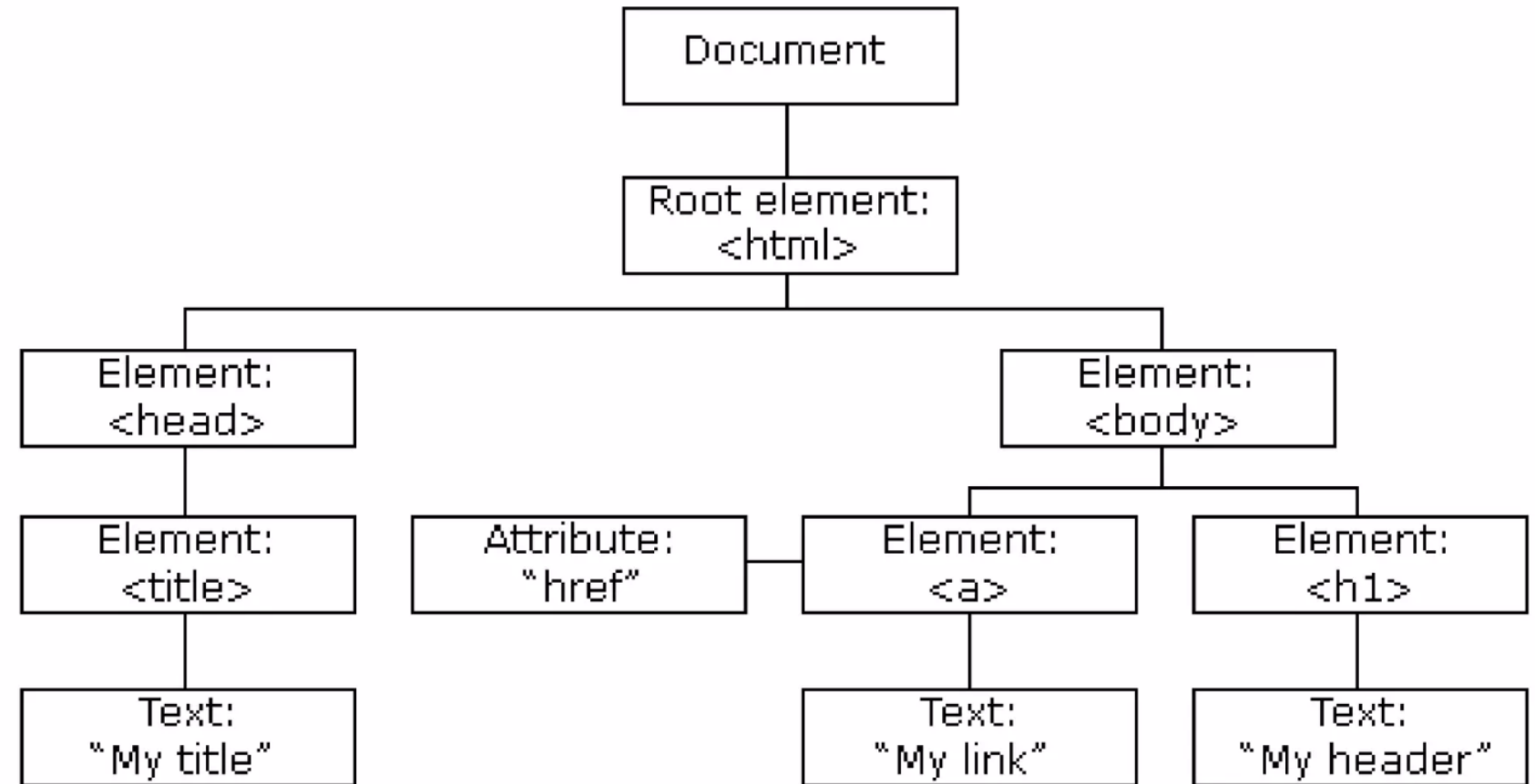
Document

Element

Element

Element

Element

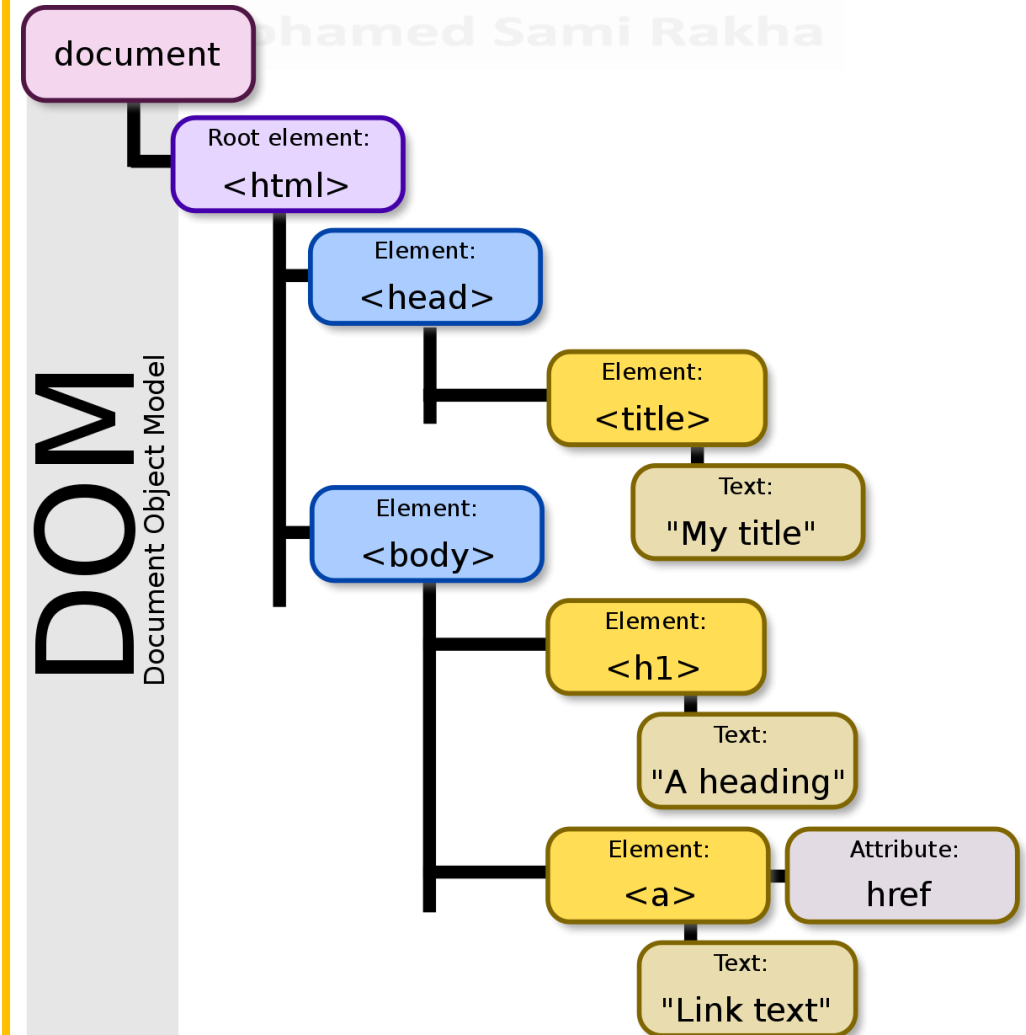


Node Types

The DOM represents an HTML document as a **hierarchical tree structure**. As a browser loads a web page, it creates a model of that page. The model is called a DOM tree, and it is stored in the browsers' memory. It consists of four main types of nodes:

- **Document Node** → root of the tree.
- **Element Nodes** → HTML tags (<html>, <body>, <p>, etc.).
- **Attribute Nodes** → attributes inside elements (class, id).
- **Text Nodes** → the actual text inside elements.

Parent-child relationships describe the structure.
Makes it easy to traverse, access, and modify the document programmatically.



DOM Tree – Simple Example

The **Document** node is always the **root**.

Each **HTML element** (like <body>, <p>) is an element node.

Text inside elements is represented separately as text nodes.

JavaScript can access any node and change its content, attributes, or even remove/add new nodes.

Dr. Mohamed Sami Rakha

```
<!DOCTYPE html>
<html>
  <head>
    <title>My Page</title>
  </head>
  <body>
    <h1>Welcome</h1>
    <p>Hello World</p>
  </body>
</html>
```

Document

- html
 - head
 - title
 - "My Page"
 - body
 - h1
 - "Welcome"
 - p
 - "Hello World"

Document

- html
 - head
 - My Page
 - title
- body
 - h1
 - Welcome
 - p
 - Hello World

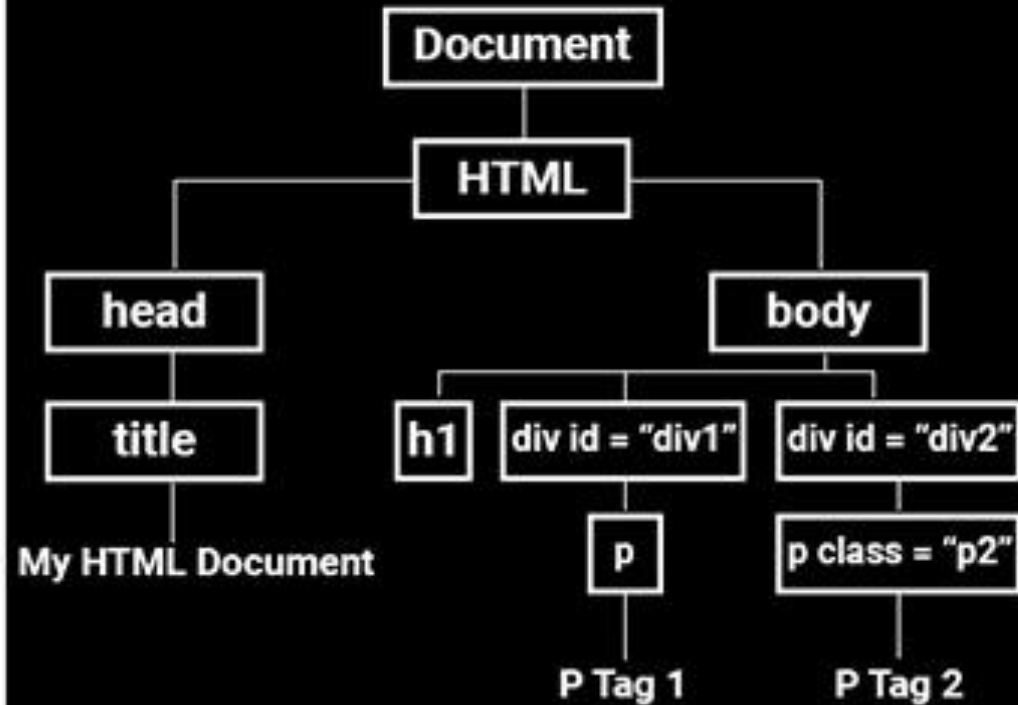
DOM Tree – Simple Example

What is Document Object Model ?

HTML Document

```
index.html x
1  <html>
2    <head>
3      <title>My HTML Document</title>
4    </head>
5
6    <body>
7      <h1>Heading</h1>
8      <div id="div1">
9        <p>P Tag 1</p>
10     </div>
11     <div id="div2">
12       <p class="p2">P Tag 2</p>
13     </div>
14   </body>
15 </html>
```

Document Object Model (DOM)



DOM and JavaScript

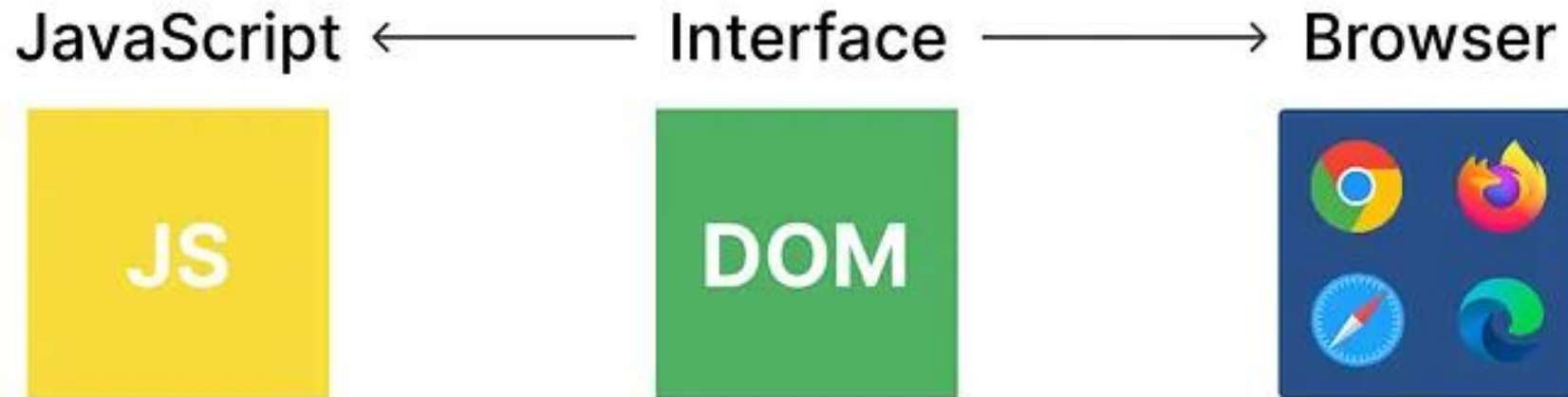
DOM and JavaScript

With the DOM Interfaces, JavaScript gets all the power it needs to create dynamic HTML:

- JavaScript can change all the HTML elements in the page.
- JavaScript can change all the HTML attributes in the page.
- JavaScript can change all the CSS styles in the page.
- JavaScript can remove existing HTML elements and attributes.
- JavaScript can add new HTML elements and attributes.
- JavaScript can react to all existing HTML events in the page.
- JavaScript can create new HTML events in the page.

Commonly Used JavaScript DOM Interfaces

Dr. Mohamed Sami Rakha



Commonly Used JavaScript DOM Interfaces

1. Document Interface (DOM)

- Entry point to the DOM.
- Provides methods to access elements (e.g., `getElementById`, `querySelectorAll`).
- Represents the whole HTML document.

2. Element Interface (DOM)

- Represents individual HTML elements (e.g., `<div>`, `<p>`).
- Allows manipulation of content (`innerText`, `innerHTML`) and styles (style property).
- Supports event handling (`addEventListener`).

3. Node Interface (DOM)

- Base interface for all DOM nodes (elements, text, attributes, comments).
- Provides navigation (`parentNode`, `childNodes`).

4. Event Interface (DOM)

- Represents events triggered by the user or browser (click, keydown, load).
- Works with `addEventListener` to respond to actions.

Window Interface (not Part of DOM!)

- Represents the browser window / global scope.
- Automatically available in all scripts.
- Note: window is the parent of document.
- It belongs to the **BOM (Browser Object Model)**
- Access to DOM → `window.document`.

Document Interface

1. Document Interface (DOM)

Dr. Mohamed Sami Rakha

- Entry point to the DOM.
- Used to access and manipulate the whole HTML document.

```
document.getElementById("myDiv");  
document.querySelectorAll("p");
```

Document Interface: Popular Functions / Properties

Function / Property	Description
<code>getElementById(id)</code>	Returns the element with the given ID.
<code>getElementsByClassName(className)</code>	Returns a live HTMLCollection of elements with the given class.
<code>getElementsByTagName(tagName)</code>	Returns a live collection of elements with the given tag (e.g., <p>).
<code>querySelector(selector)</code>	Returns the first element that matches a CSS selector.
<code>querySelectorAll(selector)</code>	Returns all elements that match a CSS selector (static NodeList).
<code>createElement(tag)</code>	Creates a new element node (e.g., <div>).
<code>createTextNode(text)</code>	Creates a new text node.
<code>write(string)</code>	Writes text/HTML directly into the document. (Rarely used today).
<code>title</code>	Gets/sets the page title.
<code>URL</code>	Returns the document's full URL.

Element Interface

2. Element interface

Dr. Mohamed Sami Rakha

- Represents a single HTML element.
- Provides methods & properties for manipulation.

```
button.innerHTML = "Hello!";  
button.setAttribute("class", "highlight");  
button.classList.add("active");
```

Note: “button” is a variable of any element fetched. ???

Element Interface: Popular Functions / Properties

Function / Property	Description
<code>getAttribute(name)</code>	Returns the value of the specified attribute.
<code>setAttribute(name, value)</code>	Sets a new value for an attribute.
<code>removeAttribute(name)</code>	Removes an attribute.
<code>classList.add(name)</code>	Adds a CSS class to the element.
<code>classList.remove(name)</code>	Removes a CSS class.
<code>classList.toggle(name)</code>	Toggles a CSS class on/off.
<code>style.property</code>	Access or modify inline styles (e.g., <code>element.style.color</code>).
<code>innerHTML</code>	Gets/sets HTML content inside the element.
<code>innerText</code>	Gets/sets visible text (ignores hidden text, respects styles).
<code>textContent</code>	Gets/sets all text content (including hidden text).

Node Interface

3. Node interface

Dr. Mohamed Sami Rakha

A generic interface for all DOM nodes (elements, text, comments).

```
button.parentNode;  
button.childNodes;  
button.removeChild(childNodes);
```

Note: “button” is a variable of any element fetched.

Node Interface: Popular Functions / Properties

Function / Property	Description
nodeType	Returns type of node (1=Element, 3=Text, etc.).
nodeName	Returns node's name (e.g., "DIV", "#text").
nodeValue	Value of a node (mainly for text/comment nodes).
parentNode	Returns the node's parent.
childNodes	Returns a live collection of all child nodes.
firstChild	Returns the first child node.
lastChild	Returns the last child node.
appendChild(node)	Adds a new child node to the end.
removeChild(node)	Removes a child node.
replaceChild(newNode, oldNode)	Replaces an existing child with a new one.

Event Interface

4. Event interface

Dr. Mohamed Sami Rakha

Deals with user or system events.
Used with **event listeners**.

```
button.addEventListener("click", (e) => {  
    console.log("Button clicked!", e);  
});
```

Note: “button” is a variable of any element fetched.

Event Interface : Popular Functions / Properties

Function / Property	Description
<code>addEventListener(type, listener)</code>	Registers a function to handle an event (e.g., "click").
<code>removeEventListener(type, listener)</code>	Removes a previously registered handler.
<code>preventDefault()</code>	Prevents the default action (e.g., prevent form submission).
<code>stopPropagation()</code>	Stops the event from bubbling up the DOM.
<code>target</code>	The element that triggered the event.
<code>type</code>	The event type (e.g., "click", "keydown").
<code>timeStamp</code>	When the event was created.
<code>currentTarget</code>	The element to which the event handler is attached.
<code>isTrusted</code>	Whether the event was triggered by the user (true) or script (false).
<code>defaultPrevented</code>	Boolean showing whether <code>preventDefault()</code> was called.

Window Interface

5. Window interface (Not a DOM interface but important to mention)

- window is not a DOM interface, it's the container for the DOM.
- it belongs to the **BOM (Browser Object Model)**.
- **Browser controls & metadata** → window.location, window.history, window.navigator.
- **Global functions** → alert(), setTimeout(), setInterval().

Hierarchy

Window (BOM)

- └ Document (DOM root)
 - └ Elements / Nodes
 - └ Text, Attributes, etc.

// Accessing window properties

```
console.log(window.innerWidth, window.innerHeight); // viewport size
```

// Using window functions

```
window.alert("Hello!");
```

// Access DOM via window

```
console.log(window.document.title);
```

// Timer

```
window.setTimeout(() => {  
  console.log("Executed after 2 seconds");  
}, 2000);
```

Dr. Mohamed Sami Rakha

Thank You...